



API Performance Studio

User Manual

Version 0.2.0

Fire. Measure. Validate.

Table of Contents

1. Introduction
2. Installation
3. App Overview
4. Settings — Add Environments & Microservices
5. Sidebar — Select Environment & Microservice
6. Create a Feature Group
7. Add a Scenario
8. Add a Test
9. Viewing Existing Tests
10. Authentication Setup
11. Running Performance Tests
12. Analyzing Results
13. Export & Import
14. Desktop vs Web Comparison
15. Tips & Best Practices
16. Quick Reference

1. Introduction

RedfireForge is a desktop and web application for API performance testing. It lets you define HTTP tests visually, execute them with configurable concurrency, validate responses against expected schemas, and analyze the results — all without writing a single line of test script.

RedfireForge is available as a native desktop application (macOS, Windows, Linux) and as a web application running in your browser.

Key Capabilities

- Visual HTTP test builder — no scripting required
- Hierarchical organization: Feature Groups → Scenarios → Tests
- OAuth2, Bearer Token, Basic Auth, and API Key authentication
- Global auth profiles with 4-tier inheritance
- Sequential, Parallel, and Ramp-up execution modes
- Real-time execution progress and result metrics
- Export/Import with intelligent conflict resolution
- Drag-and-drop test organization
- Cross-platform: macOS, Windows, Linux, and Web

2. Installation

Desktop Application (Recommended)

Platform	Installer	How to Install
macOS	.dmg file	Open DMG → drag RedfireForge to Applications
Windows	.msi installer	Double-click → follow the setup wizard
Linux	.AppImage / .deb	Make AppImage executable and run, or install .deb

macOS first launch: Right-click the app → Open → click "Open" in the dialog to bypass Gatekeeper security.

Web Application

If using the web version, you need Node.js installed. Run the following commands:

`npm install` — install dependencies

`npm run dev` — start the dev server

Open `http://localhost:5173` in your browser.

3. App Overview

When you launch RedfireForge, you'll see the main interface with three primary areas: the header with navigation tabs, the sidebar for filtering, and the main content area.

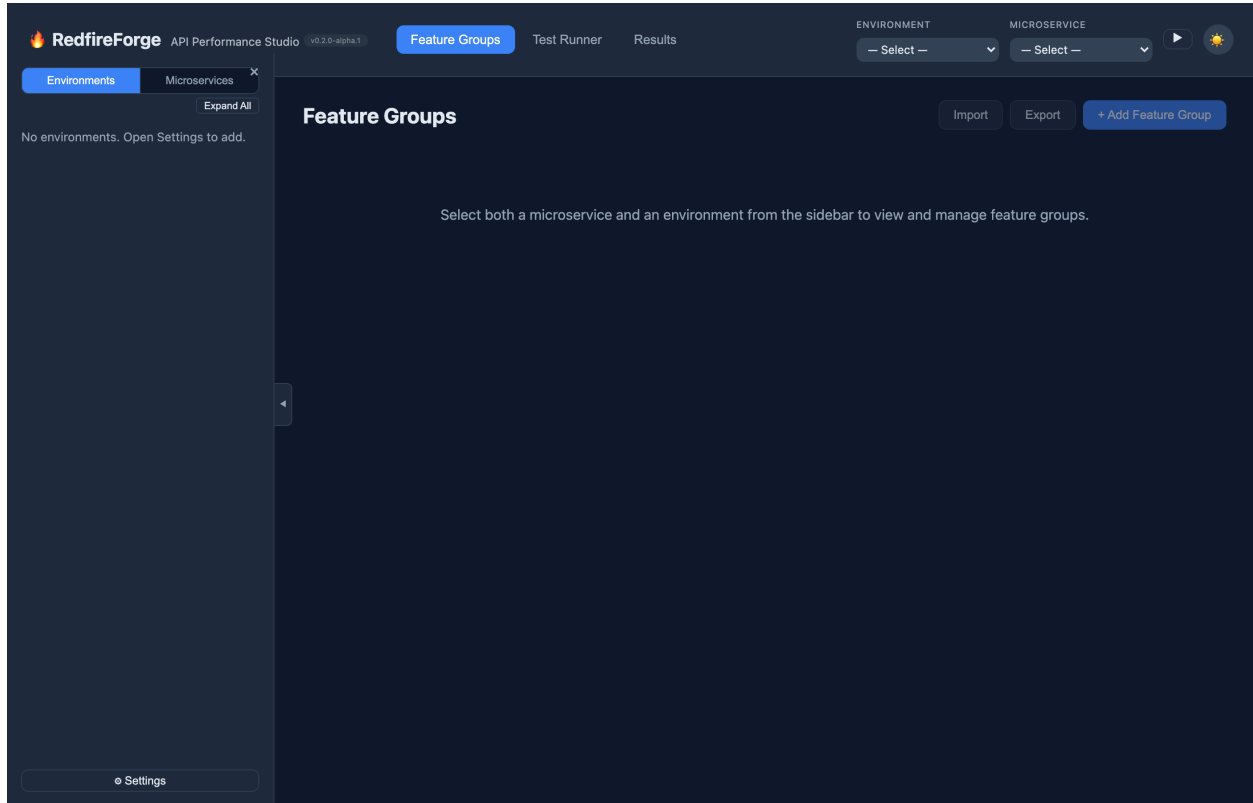


Figure 1: RedfireForge main interface on first launch

The interface has three main navigation tabs:

Tab	Purpose
Feature Groups	Create, organize, and edit your API tests
Test Runner	Configure and execute performance test runs
Results	View historical results, metrics, and charts

4. Settings — Add Environments & Microservices

Before creating tests, you need to set up at least one Environment and one Microservice. Open Settings by clicking ■ Settings in the sidebar.

Step 1: Open Settings

Click the ■ Settings button at the bottom of the sidebar.

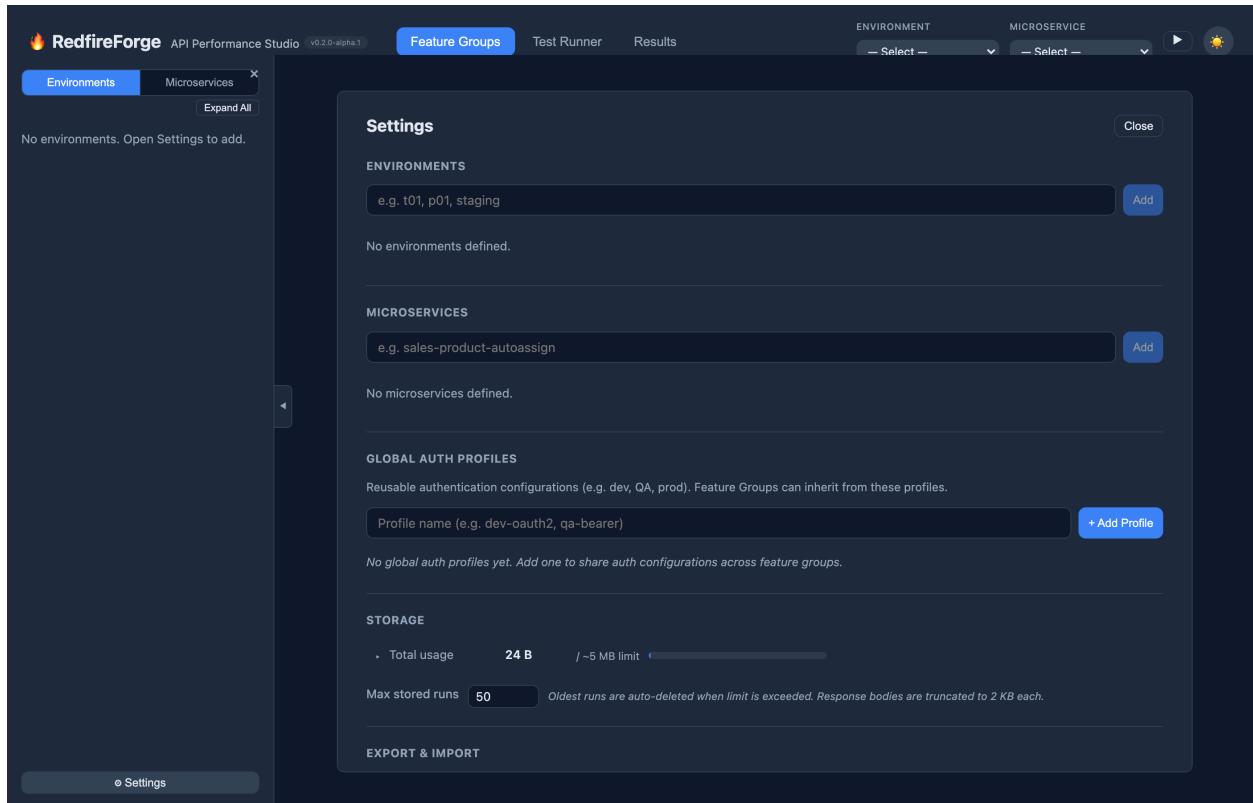


Figure 2: Settings modal — Environments and Microservices

Step 2: Add an Environment

Type an environment name (e.g., 'demo-env', 't01', 'staging') in the input field and click Add.

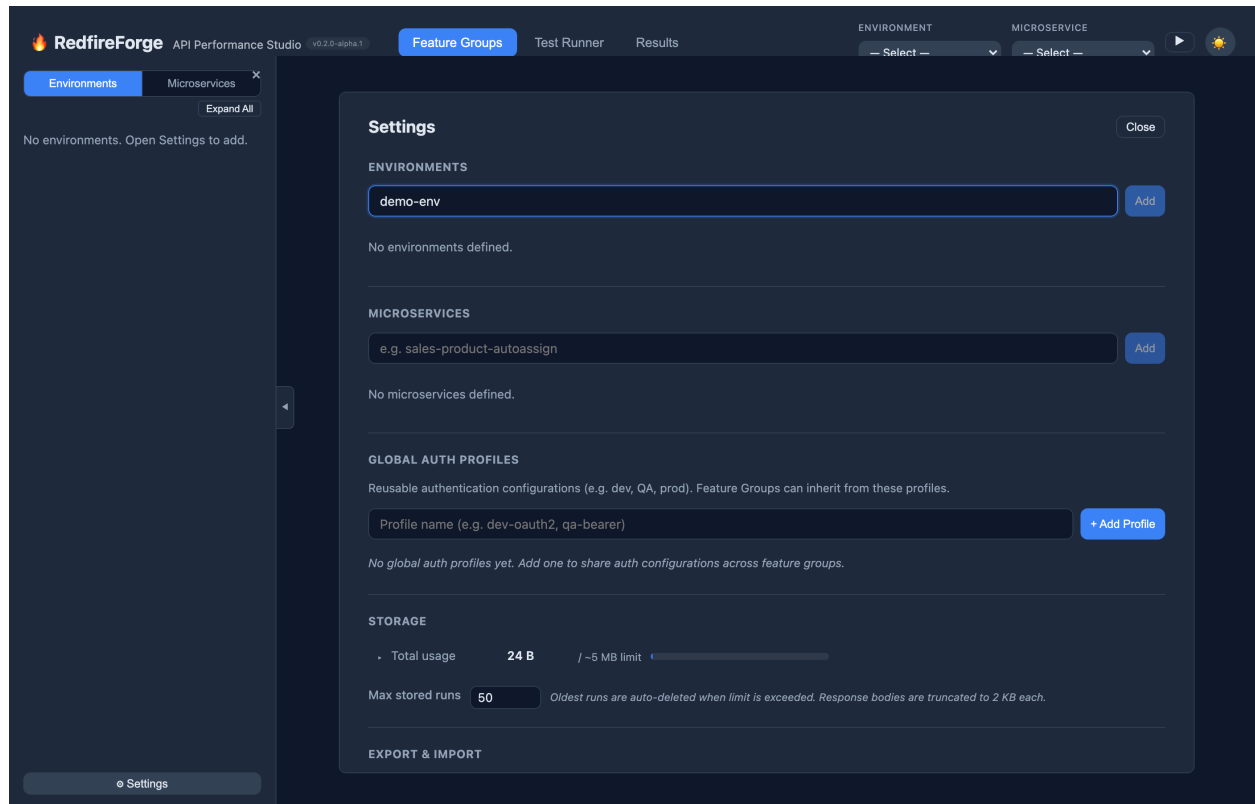


Figure 3: Typing a new environment name

Step 3: Environment Added

The environment appears as a chip below the input. You can add multiple environments.

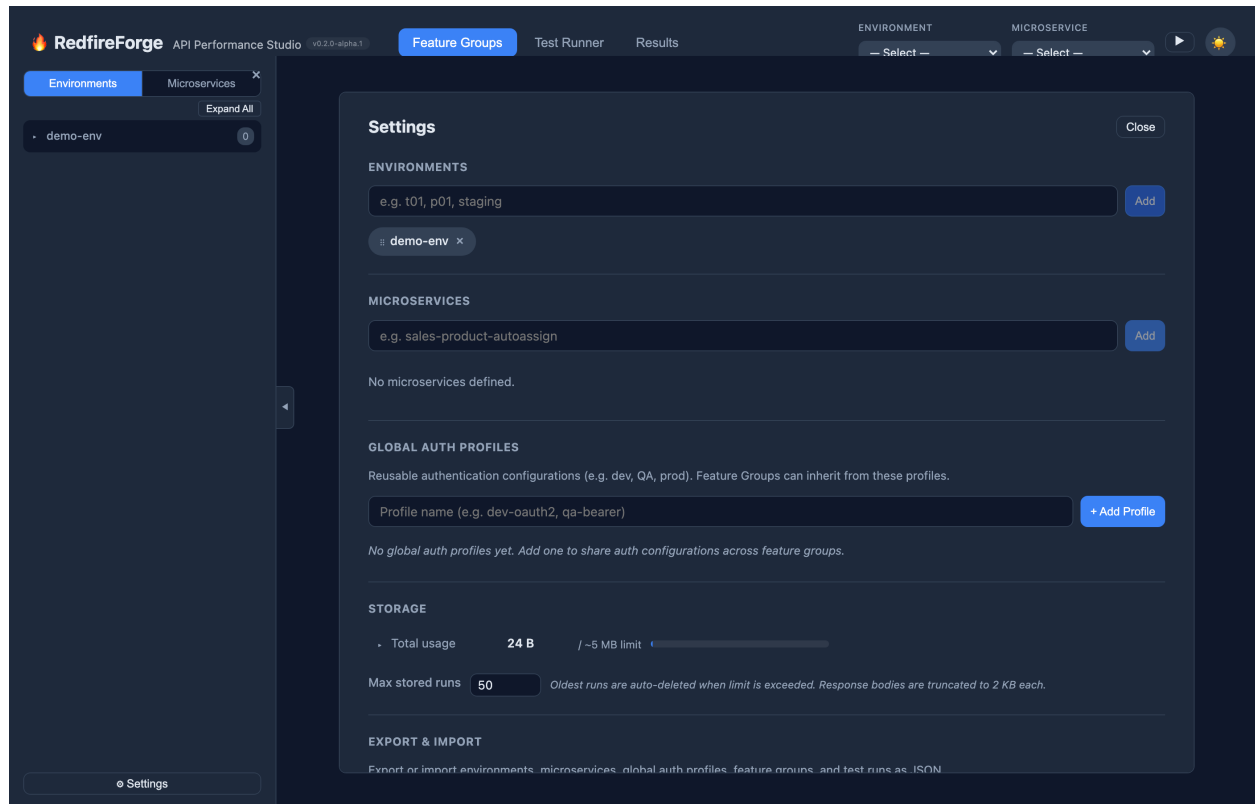


Figure 4: Environment successfully added

Step 4: Add a Microservice

Type a microservice name (e.g., 'demo-api-service') and click Add.

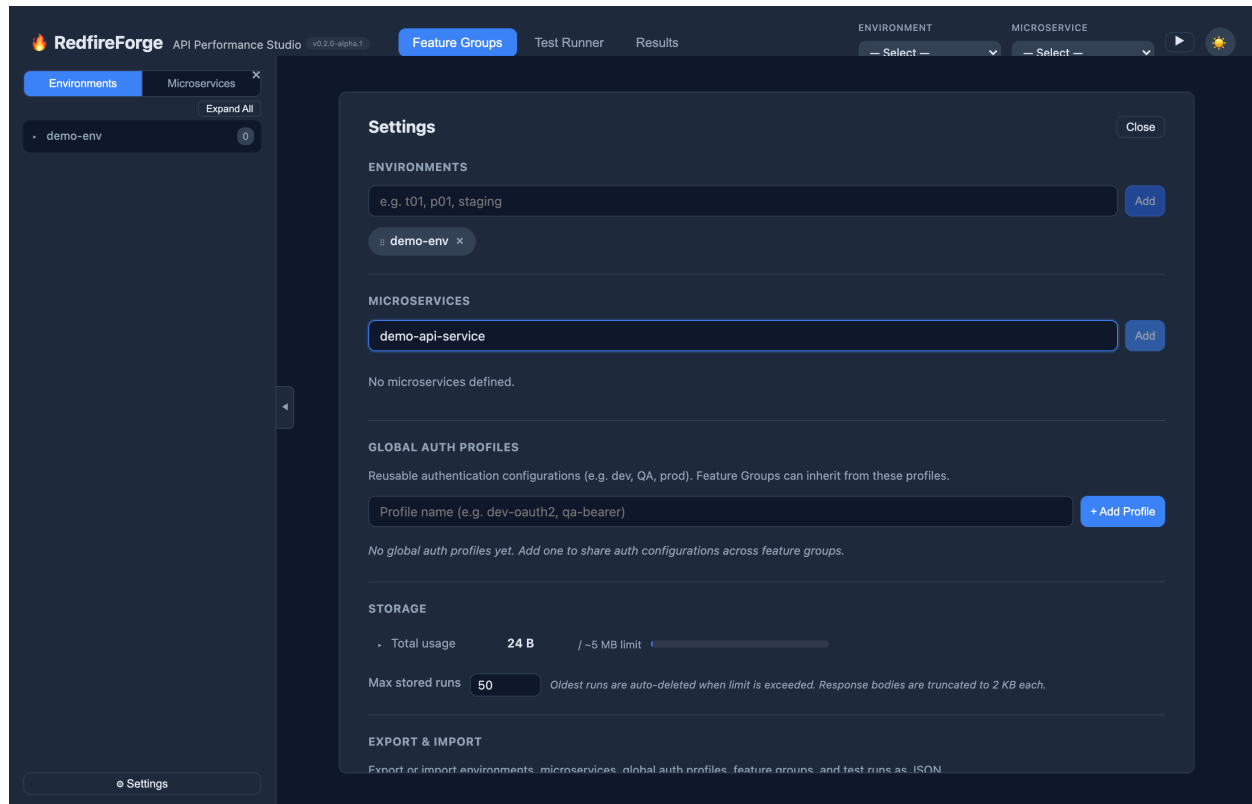


Figure 5: Typing a new microservice name

Step 5: Microservice Added

The microservice appears in the list. You can configure base URLs per environment.

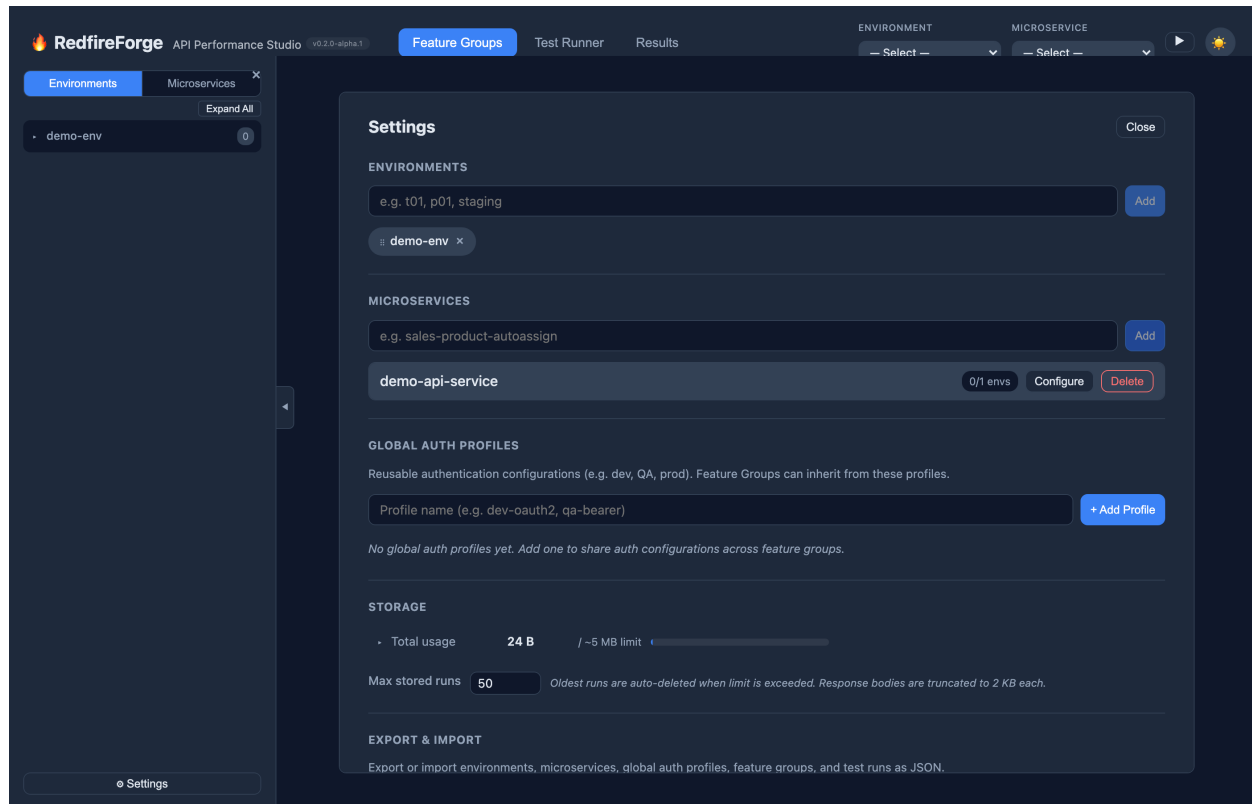


Figure 6: Microservice added with configuration options

Global Auth Profiles

Scroll down in Settings to see Global Authentication Profiles. These let you define reusable auth configurations (OAuth2, Bearer, etc.) that your tests can inherit.

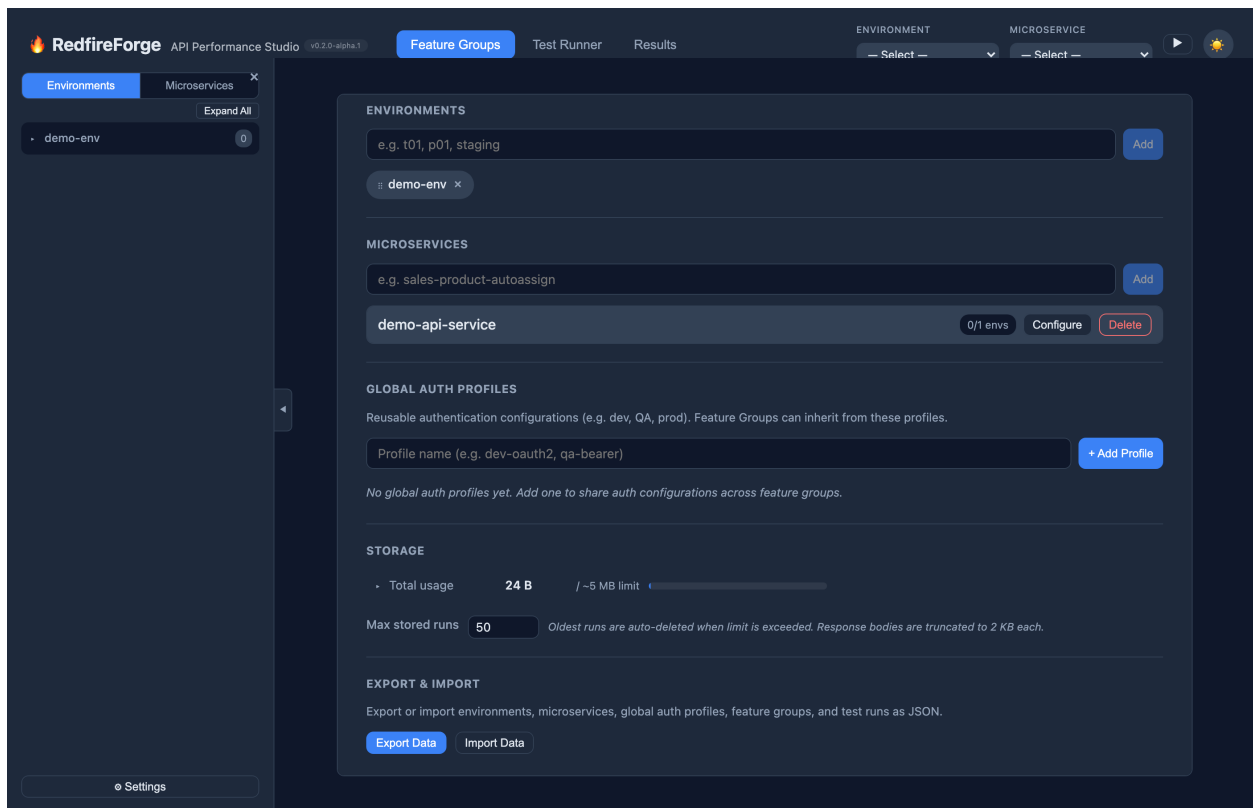


Figure 7: Global Auth Profiles section

Storage & Export/Import

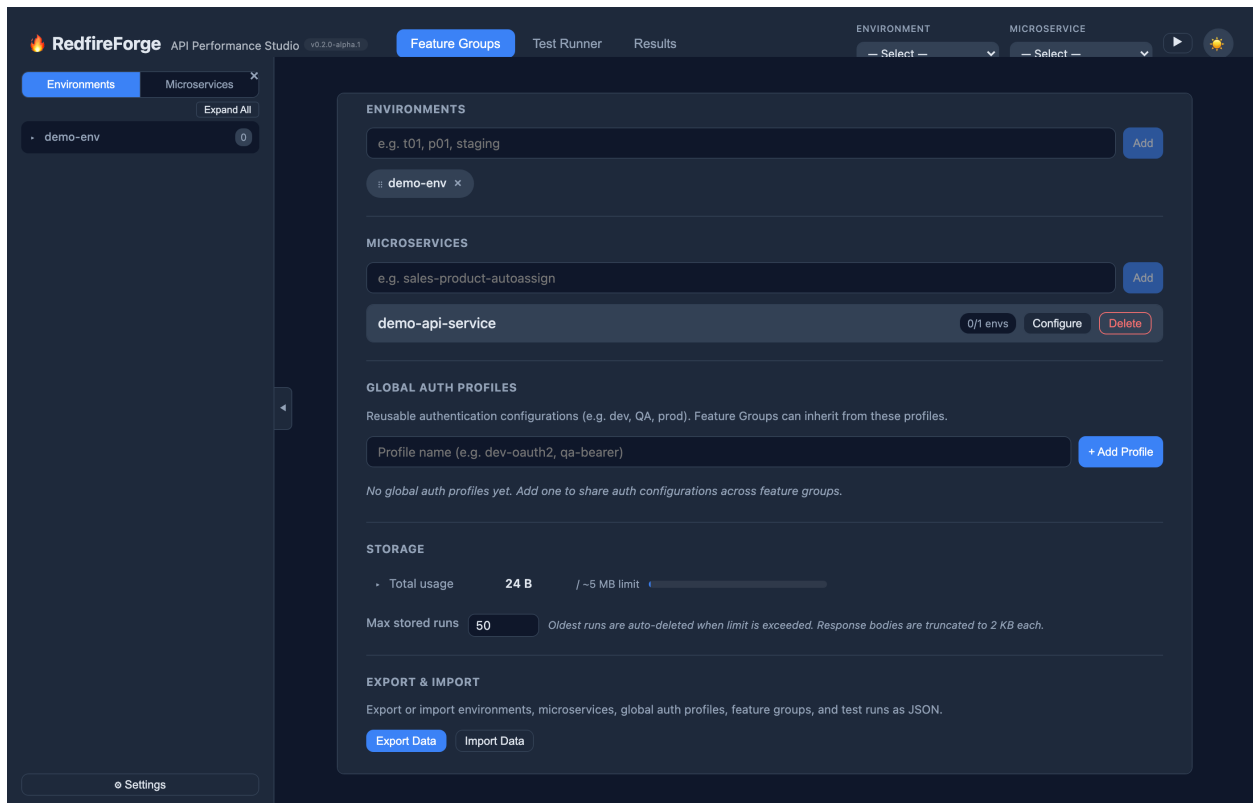


Figure 8: Storage usage and Export/Import buttons

5. Sidebar — Select Environment & Microservice

The left sidebar lets you filter Feature Groups by Environment and Microservice. You must select both before you can create or view Feature Groups.

Step 1: Click an Environment

Click on an environment name (e.g., 'demo-env') to select it. The tree expands to show microservices deployed to that environment.

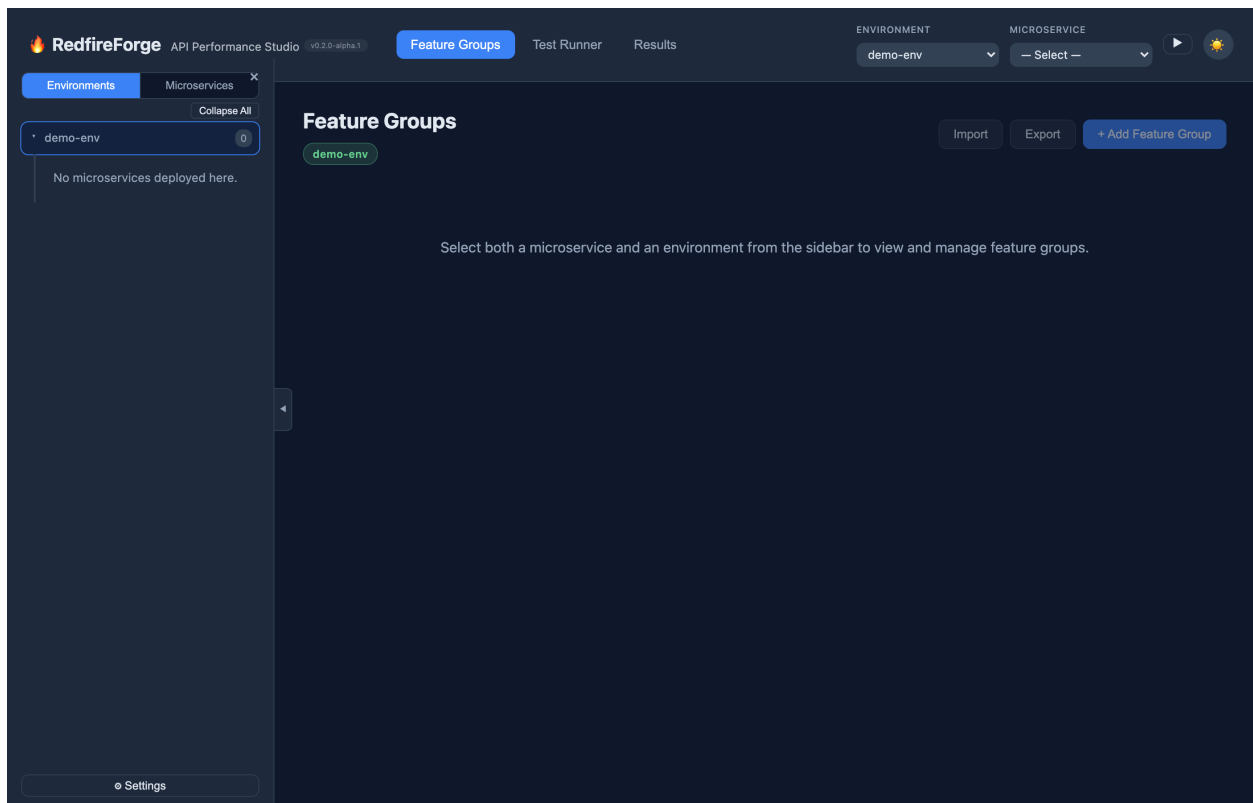


Figure 9: Clicking an environment in the sidebar

Step 2: Select a Microservice

Click on a microservice in the expanded tree to filter Feature Groups.

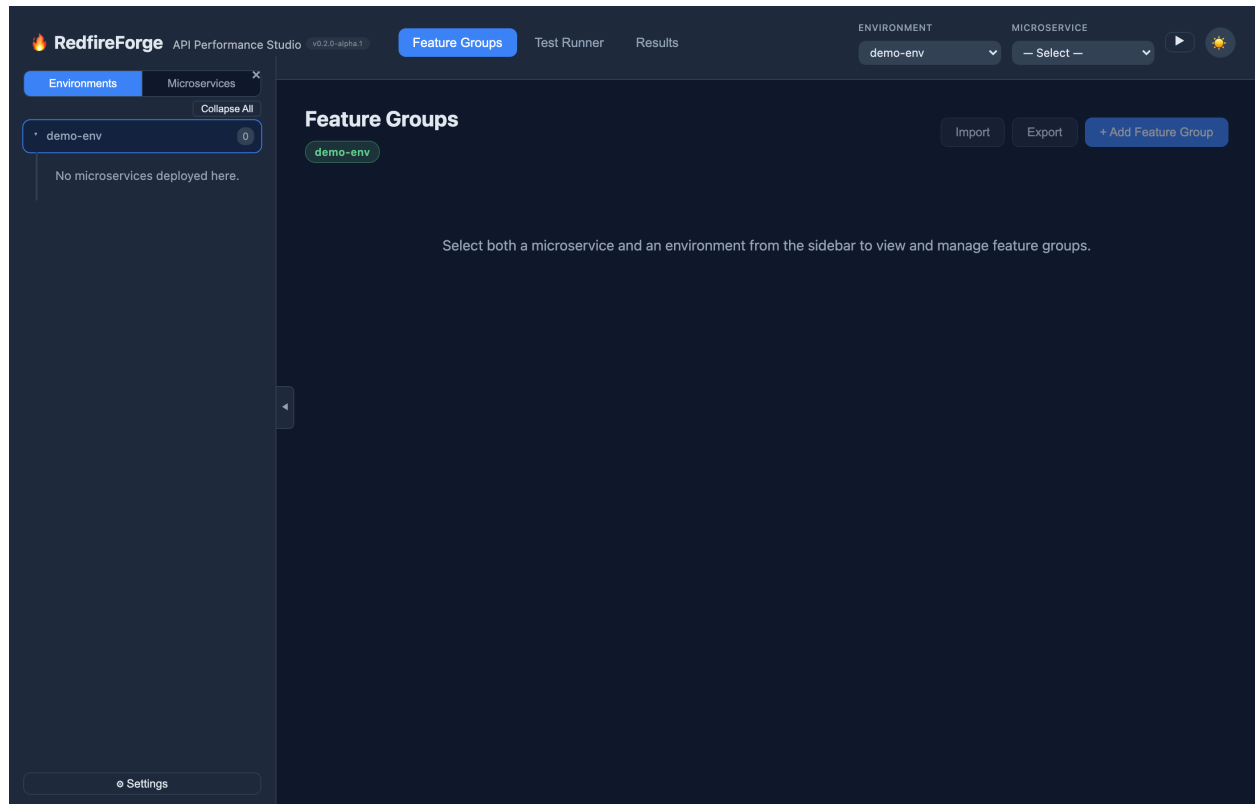


Figure 10: Microservice selected — Feature Groups now filtered

Step 3: Microservices Tab

You can also switch to the Microservices tab to browse by service first.

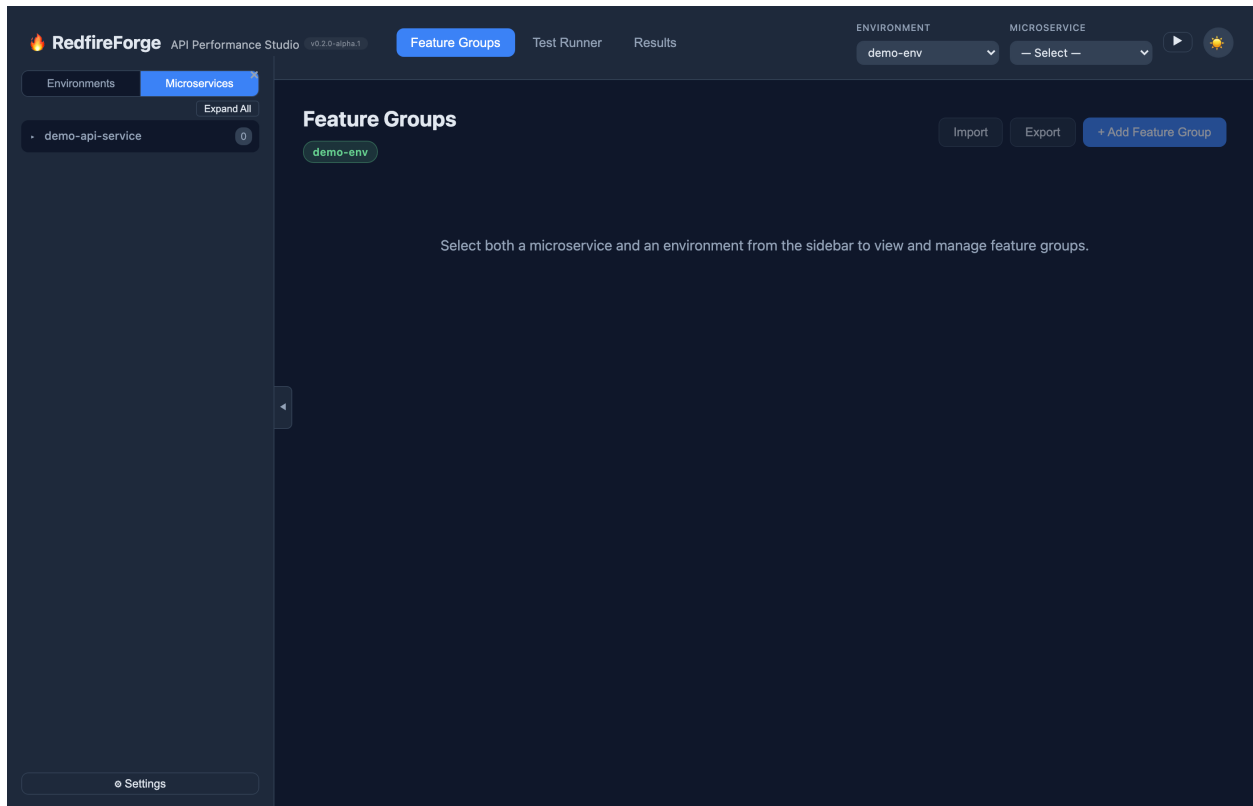


Figure 11: Sidebar Microservices tab view

6. Create a Feature Group

Feature Groups are the top-level containers for organizing your tests. You can also select the environment and microservice from the dropdowns in the header.

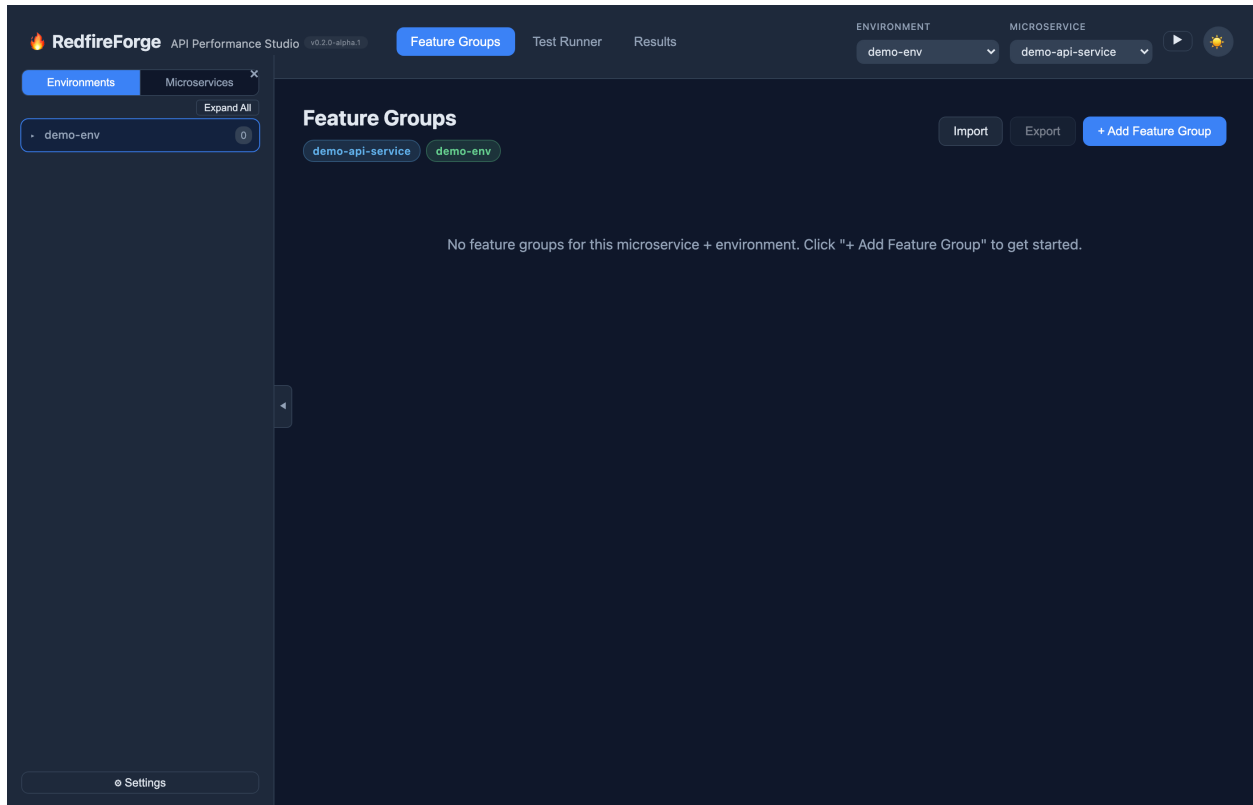


Figure 12: Environment and Microservice selected via header dropdowns

Step 1: Click '+ Add Feature Group'

An inline form appears where you can type the name.

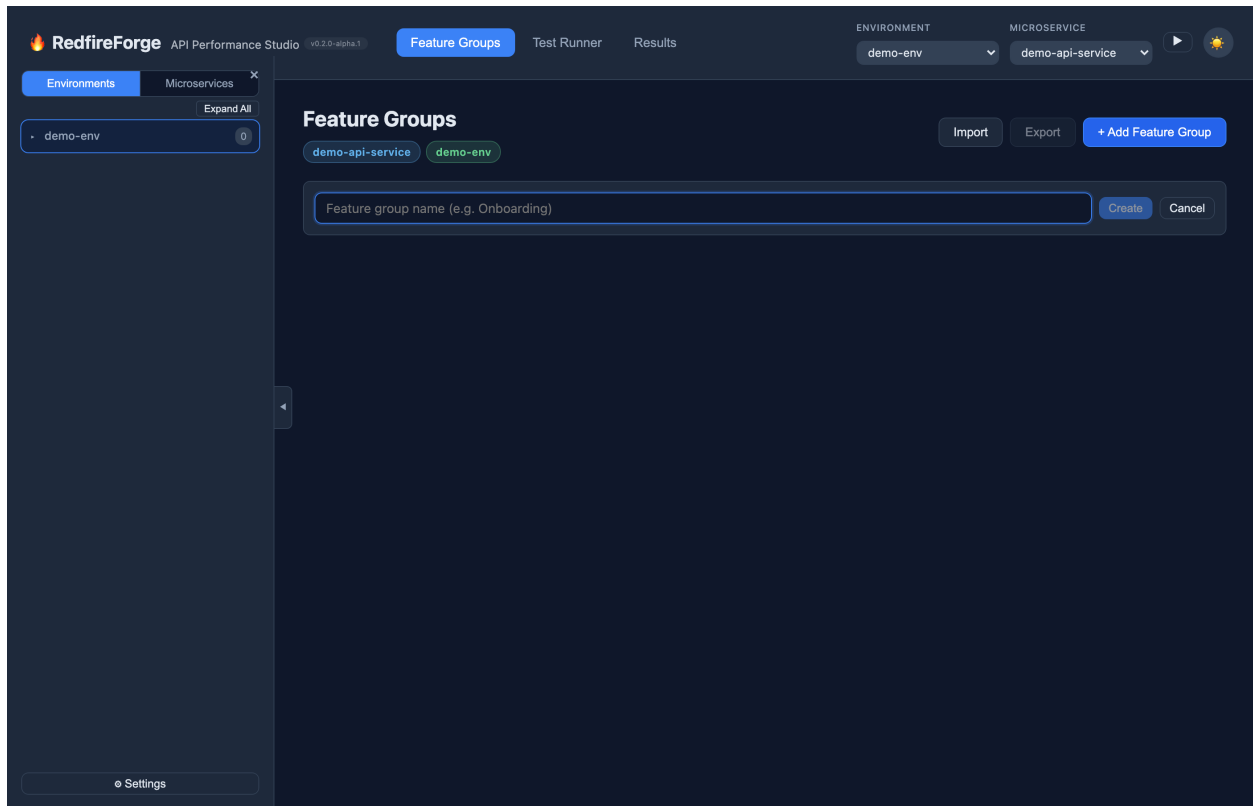


Figure 13: Feature Group naming form appears

Step 2: Type a Name and Click Create

Enter a descriptive name (e.g., 'Demo API Tests') and click Create.

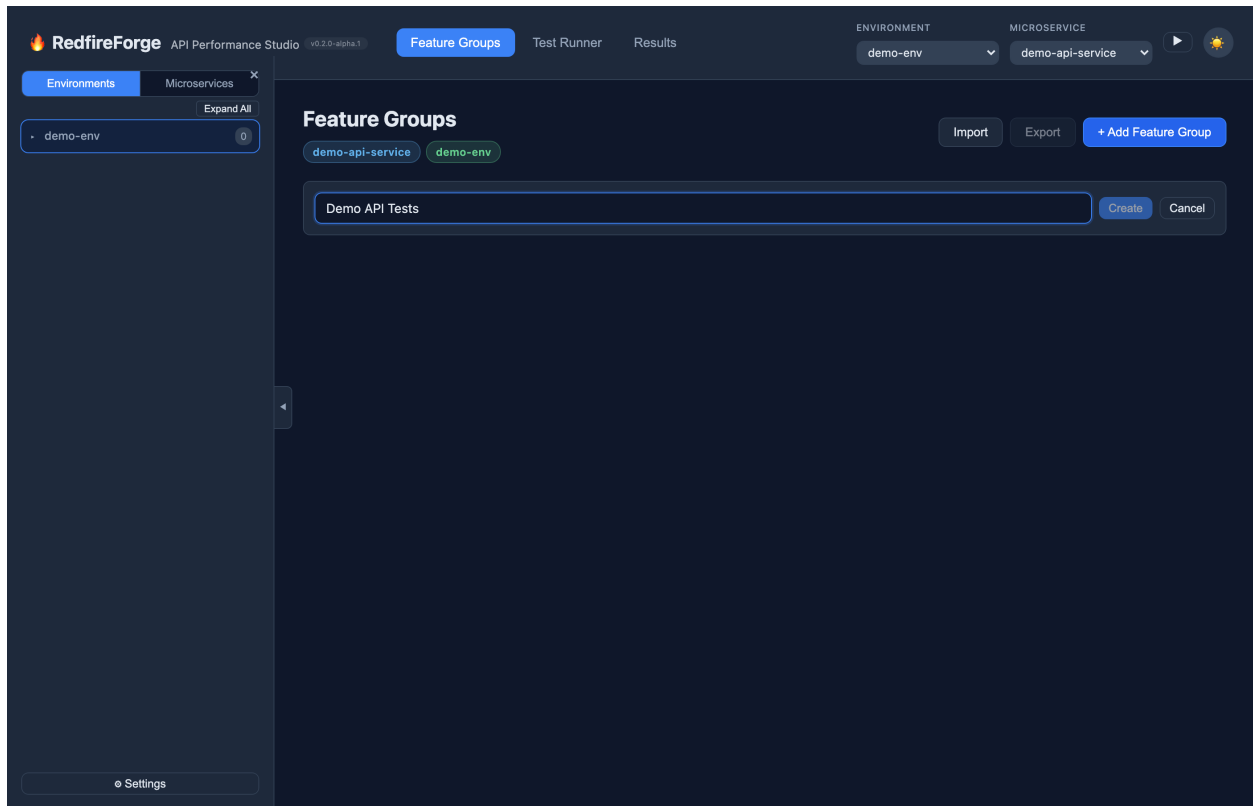


Figure 14: Typing the Feature Group name

Step 3: Feature Group Created

The new Feature Group appears in the list, expanded and ready for scenarios.

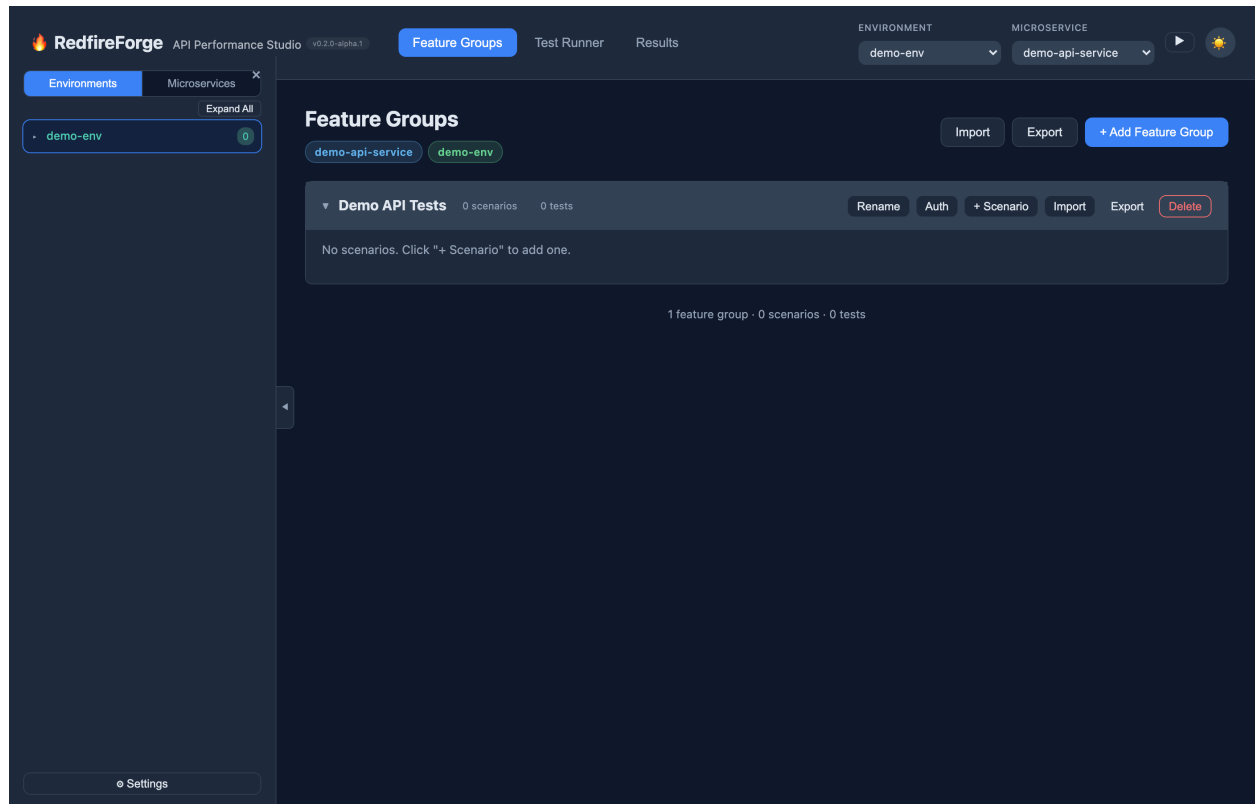


Figure 15: Feature Group created successfully

7. Add a Scenario

Scenarios group related tests within a Feature Group (e.g., 'Health Check Flow').

Step 1: Click '+ Scenario'

Inside the Feature Group, click the '+ Scenario' button.

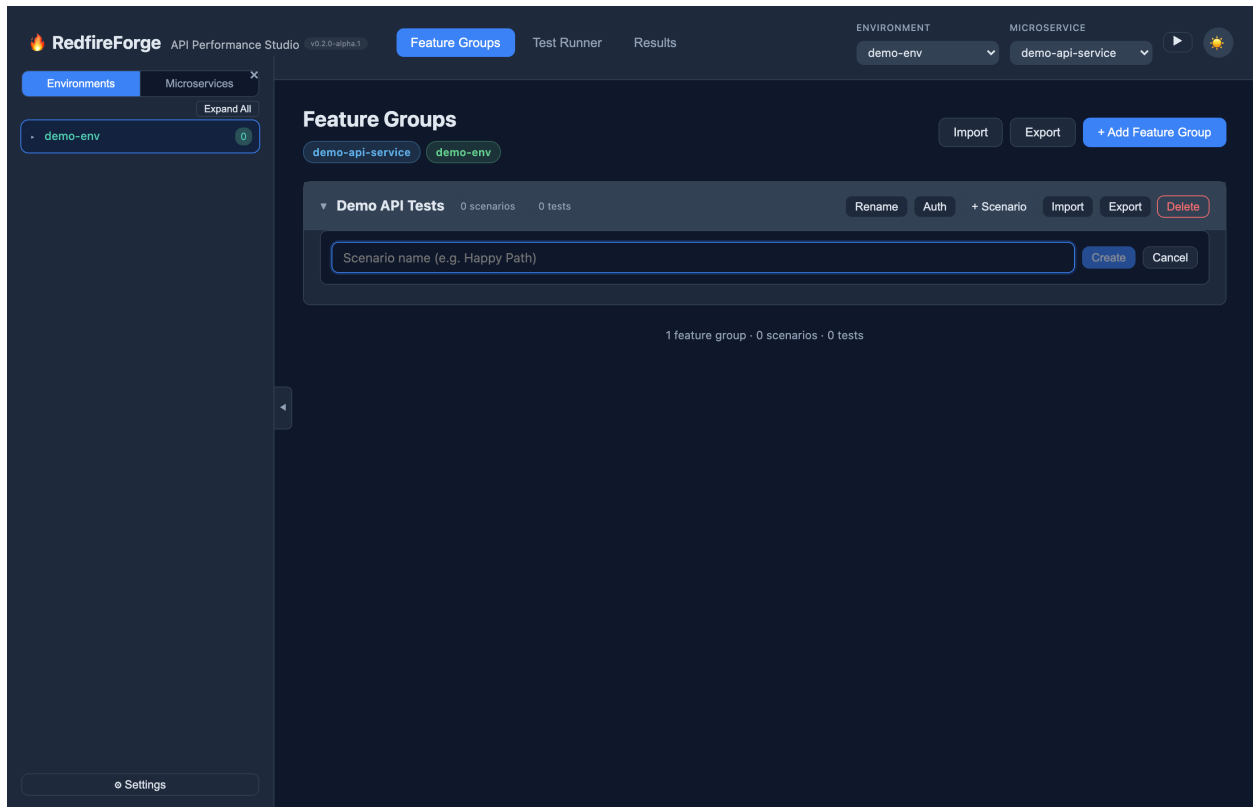


Figure 16: Scenario naming form

Step 2: Type a Name and Click Create

Enter a scenario name (e.g., 'Health Check Flow') and click Create.

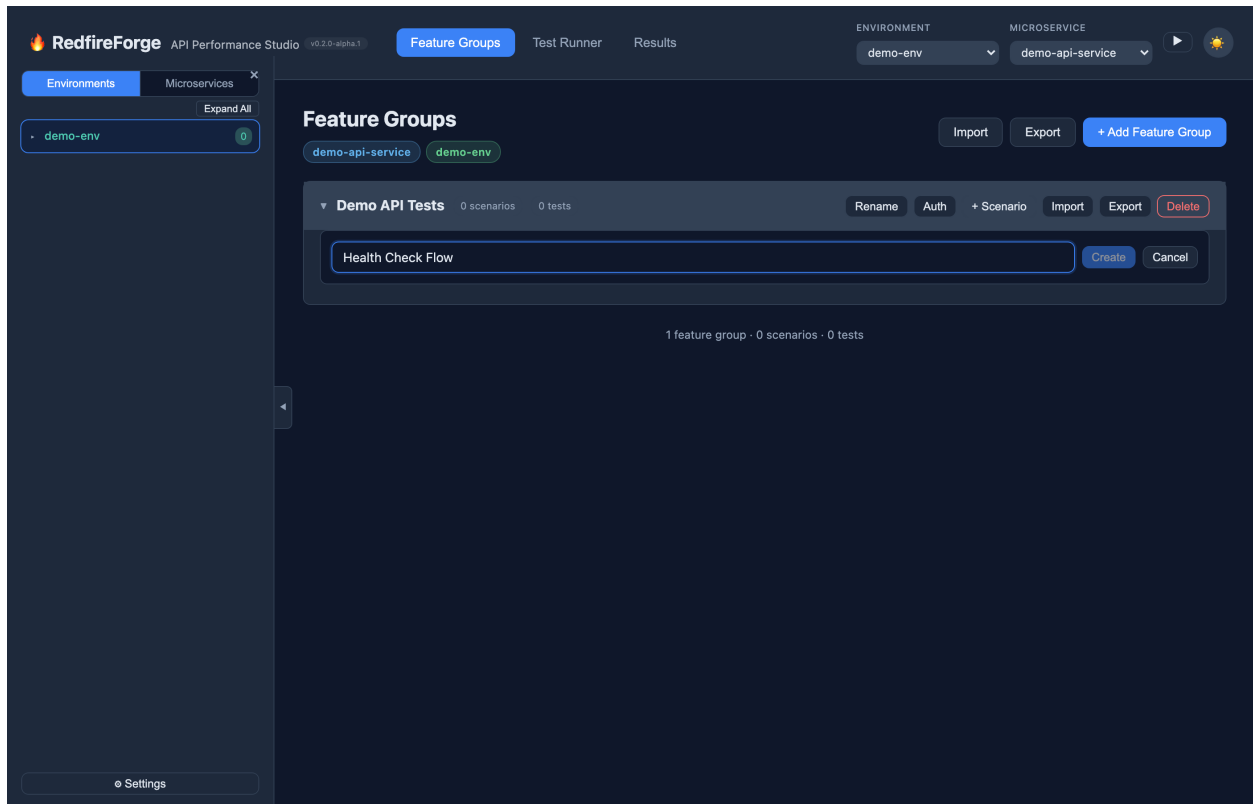


Figure 17: Typing the Scenario name

Step 3: Scenario Created

The scenario appears inside the Feature Group, ready for tests.

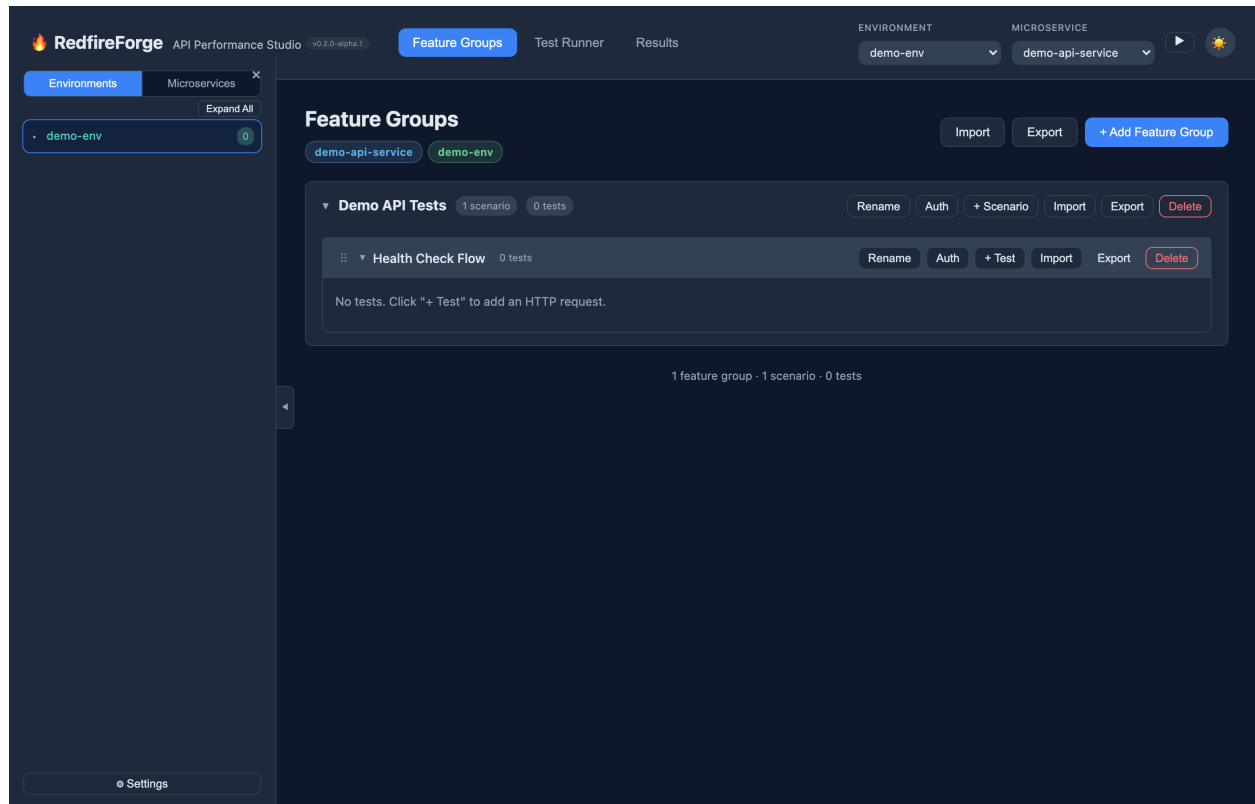


Figure 18: Scenario created with action buttons

8. Add a Test

Tests are individual HTTP requests with optional assertions.

Step 1: Click '+ Test'

Inside a Scenario, click '+ Test' to open the test editor.

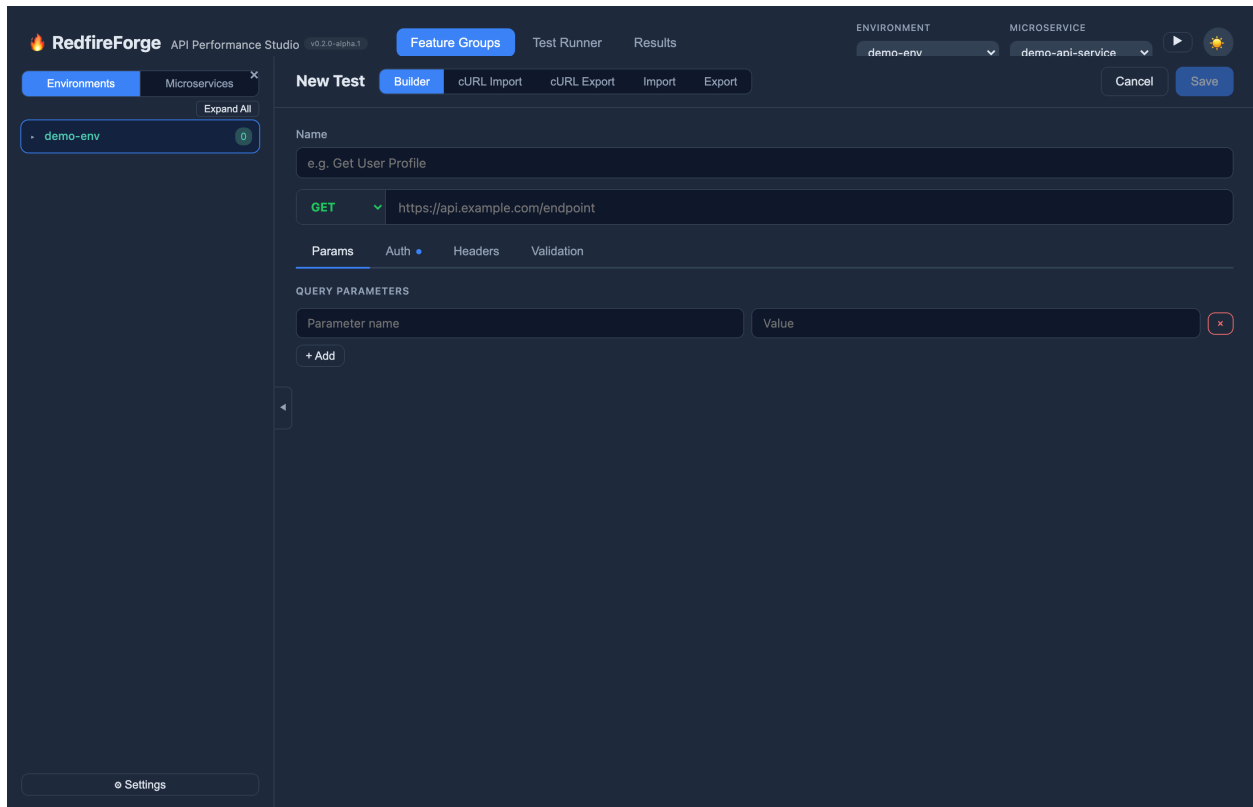


Figure 19: Test editor opens in a split panel

Step 2: Fill in Name and URL

Enter a test name (e.g., 'GET Health Check') and the request URL (e.g., 'https://httpbin.org/get'). Select the HTTP method.

The screenshot displays the RedfireForge API Performance Studio interface. The top navigation bar includes 'Feature Groups', 'Test Runner', and 'Results'. The 'ENVIRONMENT' dropdown is set to 'demo-env', and the 'MICROSERVICE' dropdown is set to 'demo-api-service'. The left sidebar shows 'Environments' with 'demo-env' selected. The main panel is titled 'New Test' and has tabs for 'Builder', 'cURL Import', 'cURL Export', 'Import', and 'Export'. The 'Builder' tab is active, showing a form to configure a new test. The 'Name' field contains 'GET Health Check'. The 'HTTP Method' dropdown is set to 'GET', and the 'URL' field contains 'https://httpbin.org/get'. Below these fields are tabs for 'Params', 'Auth', 'Headers', and 'Validation'. The 'Params' tab is active, showing a table for 'QUERY PARAMETERS' with columns for 'Parameter name' and 'Value'. There is an '+ Add' button below the table. At the bottom left, there is a 'Settings' button.

Parameter name	Value
----------------	-------

Figure 20: Test name and URL configured

Step 3: Configure More Fields

Scroll down to add headers, request body, expected status code, and response validation rules.

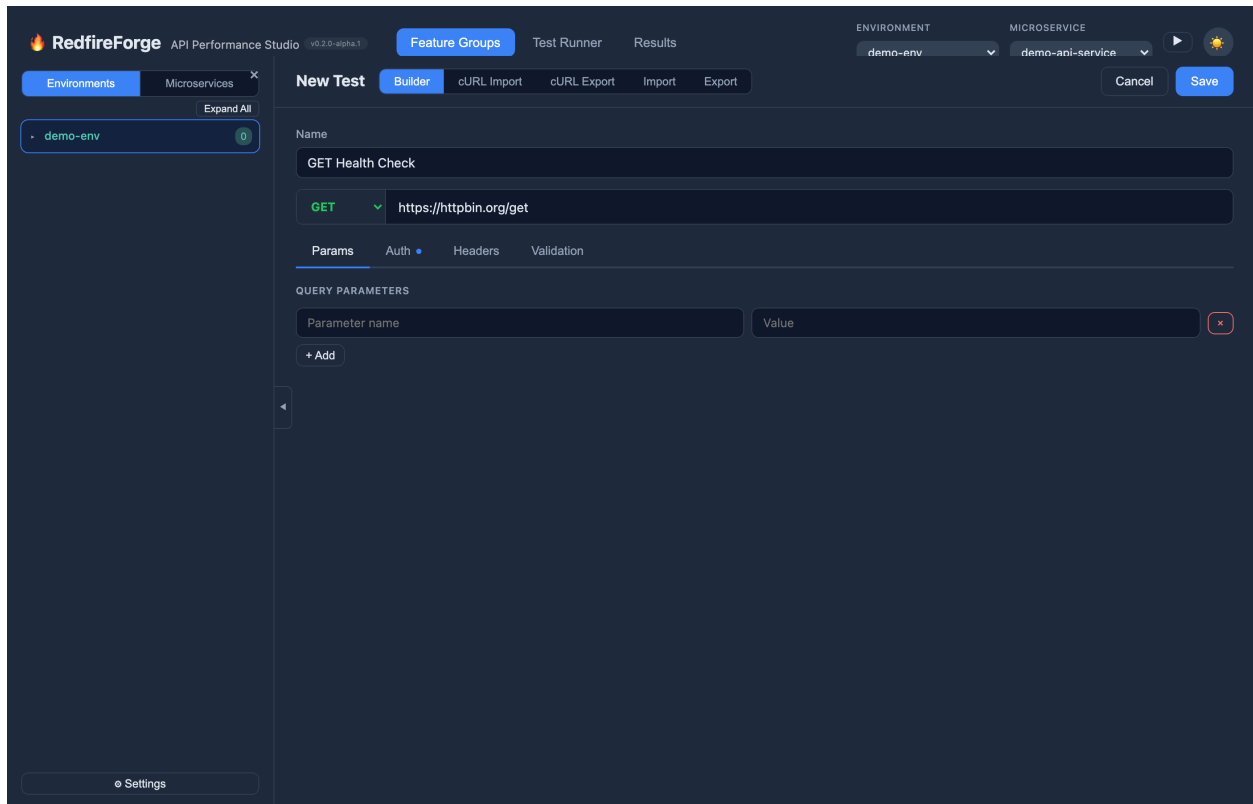


Figure 21: Additional test configuration options

Step 4: Save the Test

Click Save to add the test to the scenario.

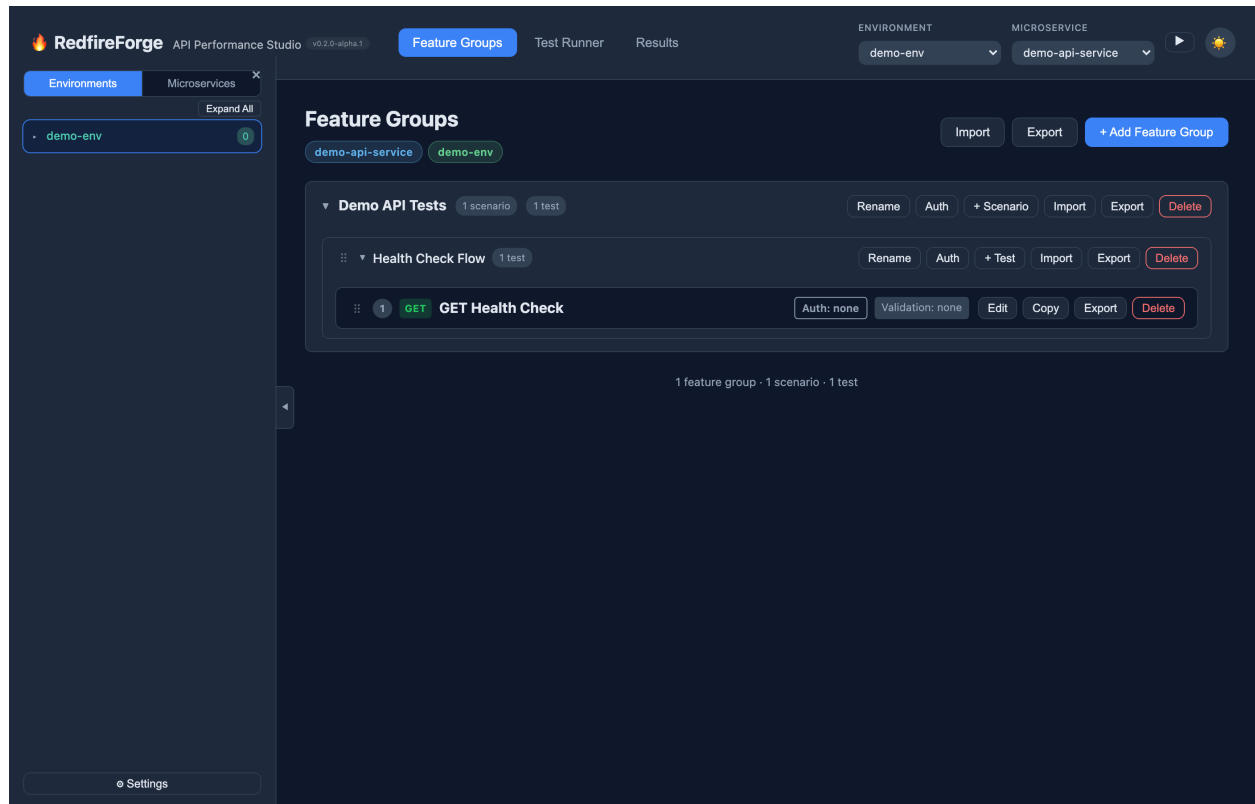


Figure 22: Test saved and visible in the scenario

Test Configuration Fields

Field	Description
Name	A descriptive name for the test
Method	HTTP method: GET, POST, PUT, DELETE, PATCH
URL	The full request URL
Headers	Key-value pairs for request headers
Body	Request body (for POST/PUT/PATCH)
Expected Status	The expected HTTP status code (e.g., 200, 201)
Validation	JSON response validation rules

Auth	Override authentication for this specific test
------	--

9. Viewing Existing Tests

Once you have Feature Groups with scenarios and tests, you can browse and expand them to see all your test configurations.

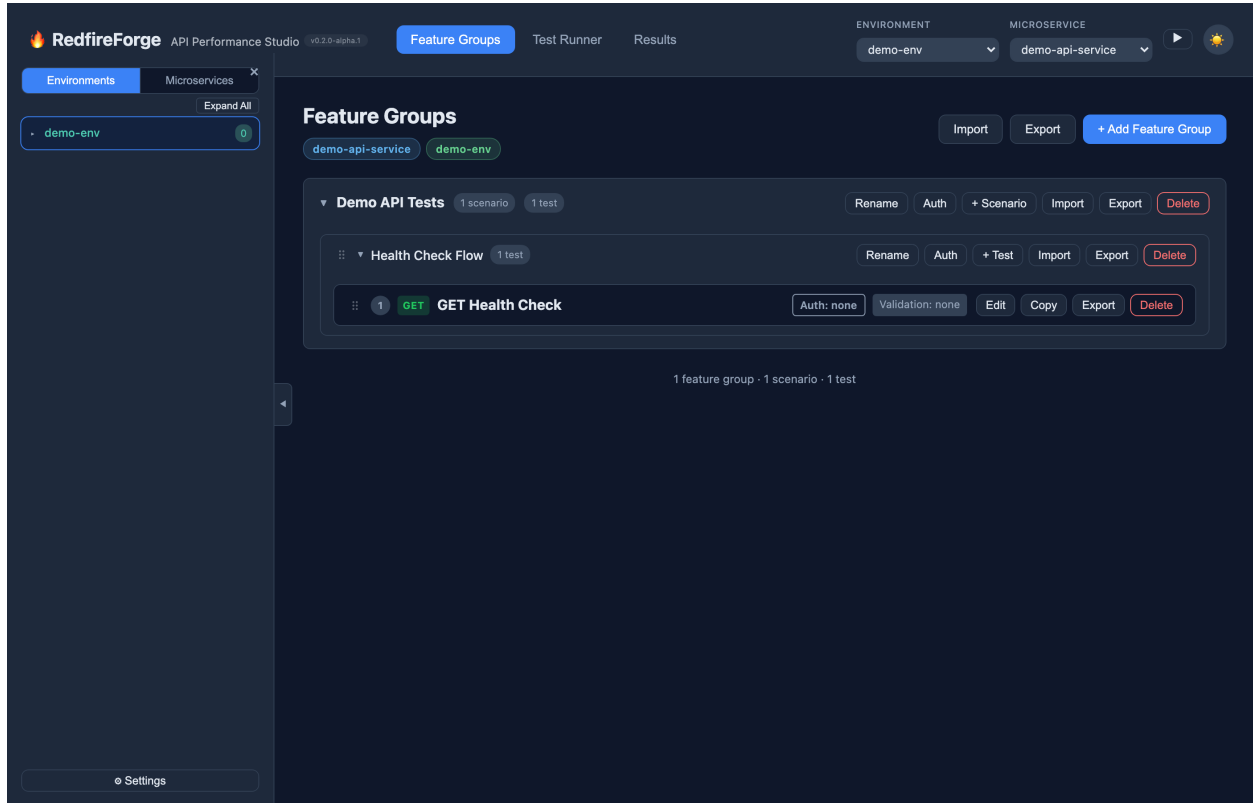


Figure 23: Feature Groups with existing test data

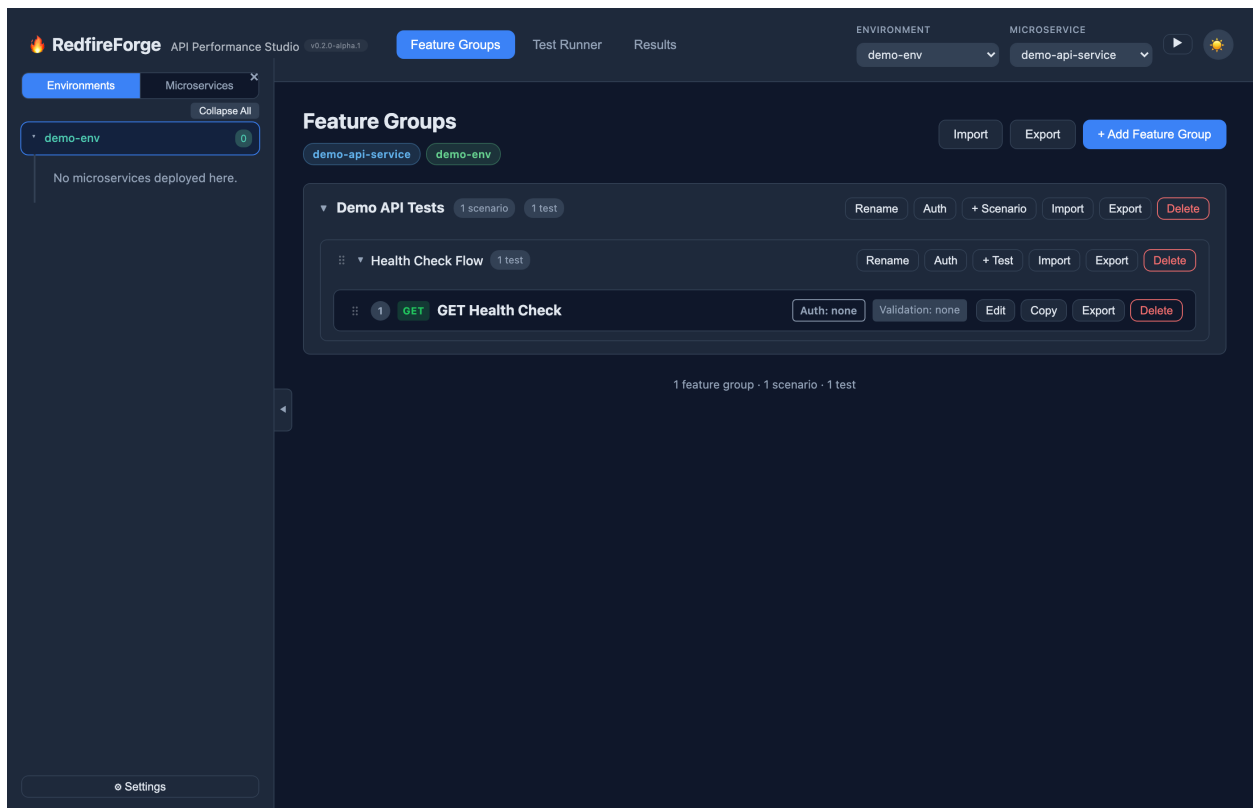


Figure 24: All Feature Groups expanded showing scenarios and tests

10. Authentication Setup

RedfireForge supports a 4-tier authentication inheritance model. Configure auth once at the top level and let it cascade down:

Level	Inherits From	Override?
Global Auth Profile	—	Base configuration
Feature Group	Global Profile	Can override
Scenario	Feature Group	Can override
Test	Scenario	Can override

Supported Auth Types

- **OAuth2 (Client Credentials)** — Token URL, Client ID, Client Secret, Scope
- **Bearer Token** — Static token value
- **Basic Auth** — Username and password
- **API Key** — Key name, value, sent in header or query

Use the "Verify" button to test your credentials before running tests.

11. Running Performance Tests

Switch to the Test Runner tab to configure and execute performance tests.

Step 1: Open Test Runner Tab

Click the 'Test Runner' tab in the header navigation.

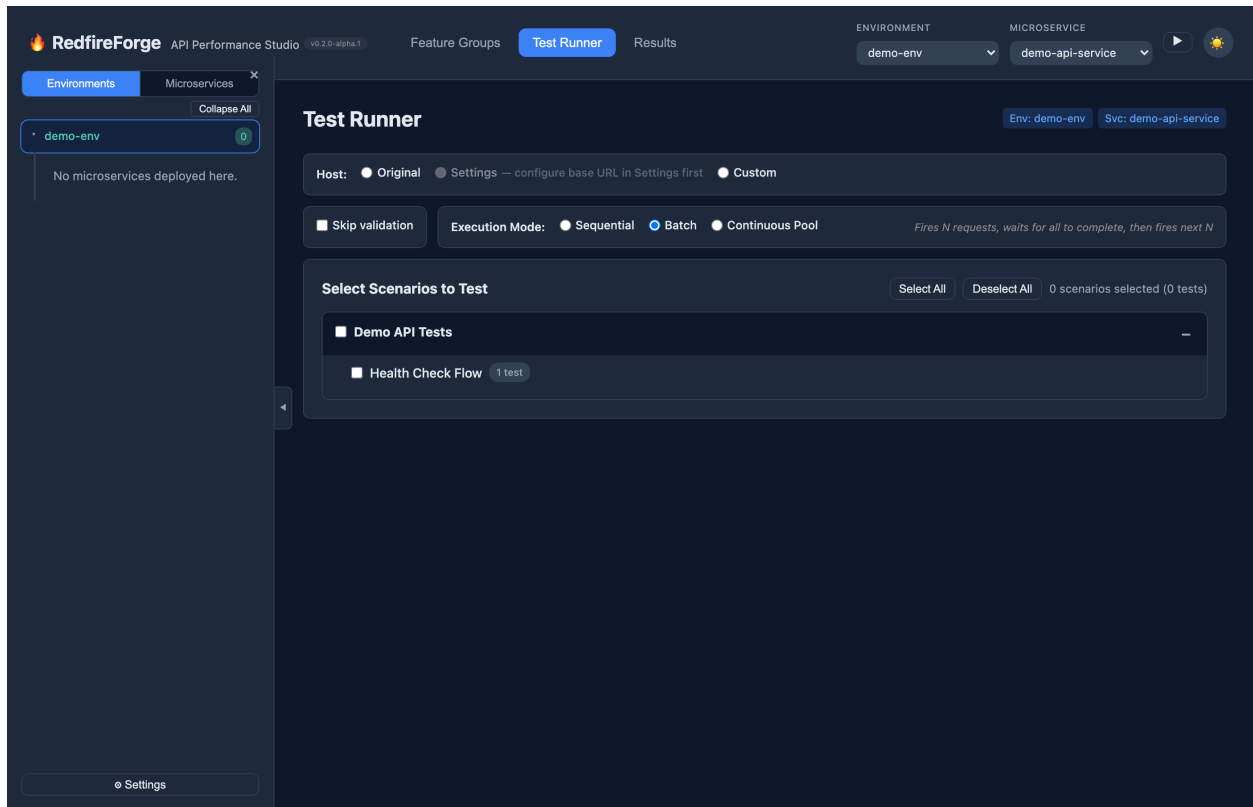


Figure 25: Test Runner tab — select tests to run

Step 2: Configure the Test Run

Select a Feature Group or Scenario to test. Set the number of iterations and choose the execution mode.

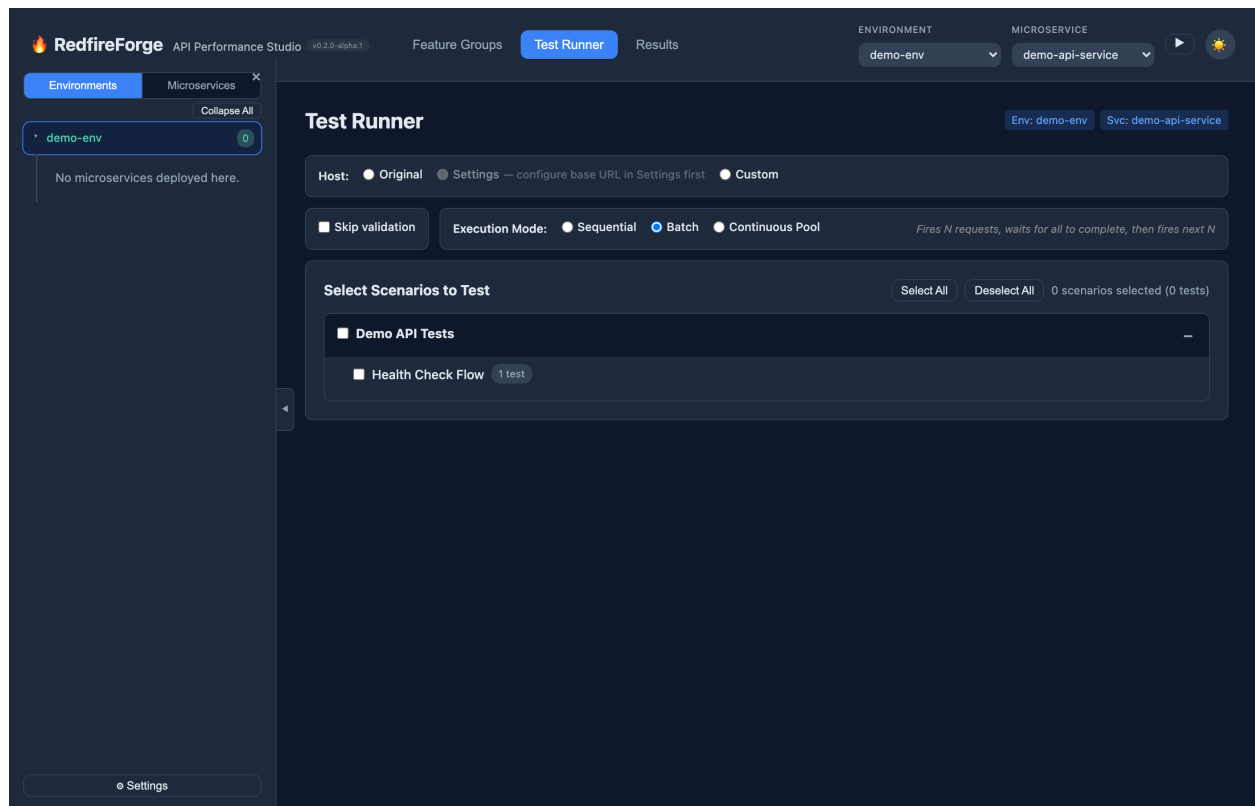


Figure 26: Test Runner configured and ready

Execution Modes

Mode	Behavior	Best For
Sequential	One request at a time	Baseline measurements
Parallel	All requests at once	Load testing
Ramp-up	Gradually increase concurrency	Stress testing

Step 3: Click Run Test

Press the  Run Test button. Watch real-time progress as tests execute. Results are automatically saved when complete.

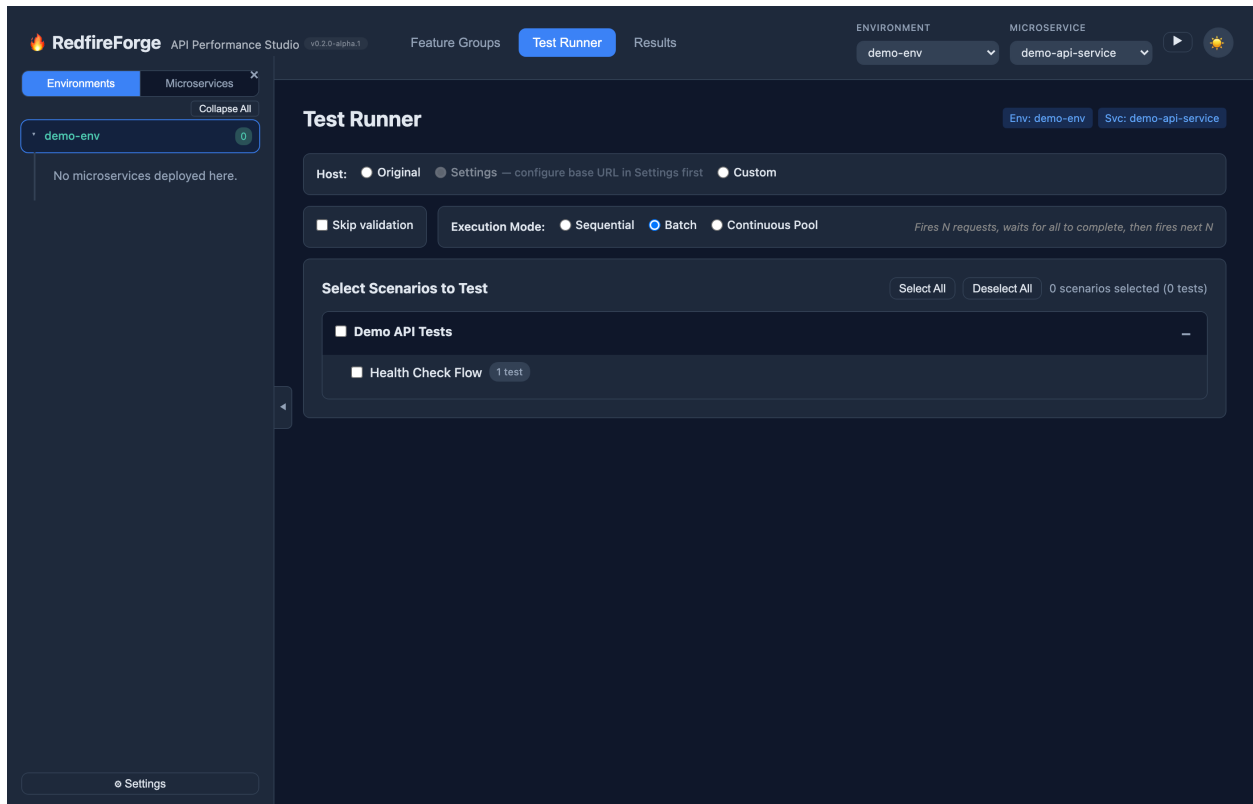


Figure 27: Run Test button — click to start execution

12. Analyzing Results

After tests complete, switch to the Results tab to view detailed metrics.

Step 1: Open Results Tab

Click the 'Results' tab to see all historical test runs.

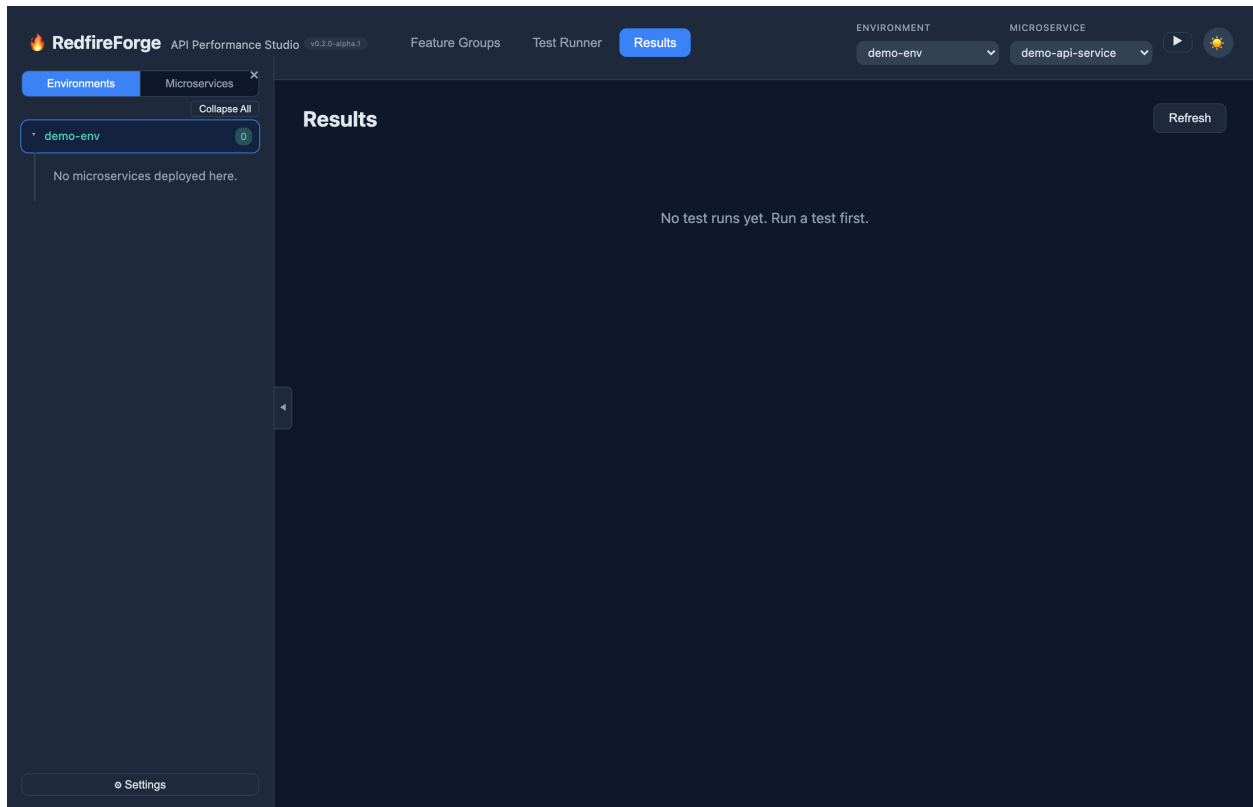


Figure 28: Results Dashboard — historical test runs

Step 2: View Detailed Metrics

Click on a test run to see response time distributions, error rates, and per-request details. Scroll down for more data.

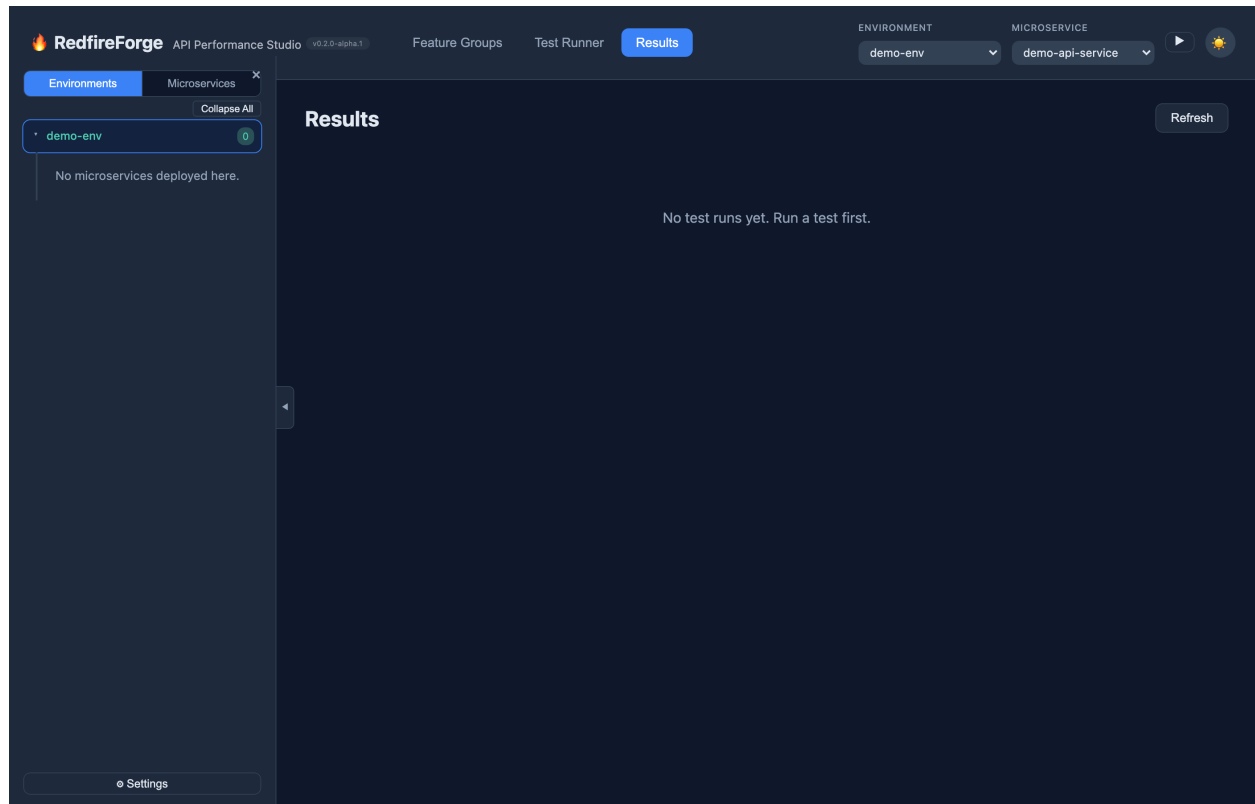


Figure 29: Detailed metrics and charts

Available Metrics

- **Average Response Time** — mean of all request durations
- **Min / Max** — fastest and slowest responses
- **P95 / P99** — 95th and 99th percentile response times
- **Error Rate** — percentage of failed requests
- **Status Code Distribution** — breakdown by HTTP status

You can export results as JSON or CSV for further analysis in spreadsheets.

13. Export & Import

Export Center

Access via Settings → Export Data. Export your entire configuration or select specific items. Files are saved to **Documents/RedfireForge/** by default.

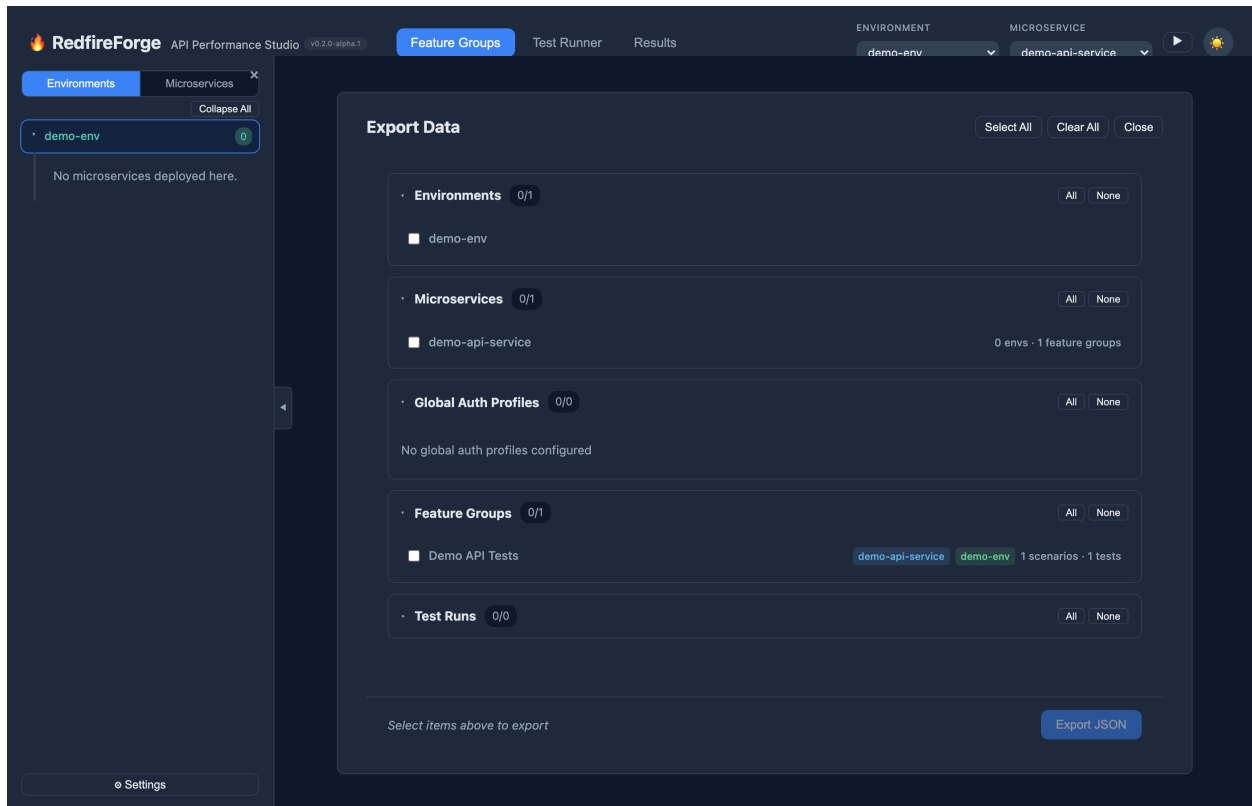


Figure 30: Export Center — select items to export

Exported files use a standardized naming convention:

```
{environment}-{microservice}-{level}-{name}-{timestamp}.json
```

Import Center

Access via Settings → Import Data. Select a previously exported .json file. RedfireForge will detect conflicts and let you resolve them per item.

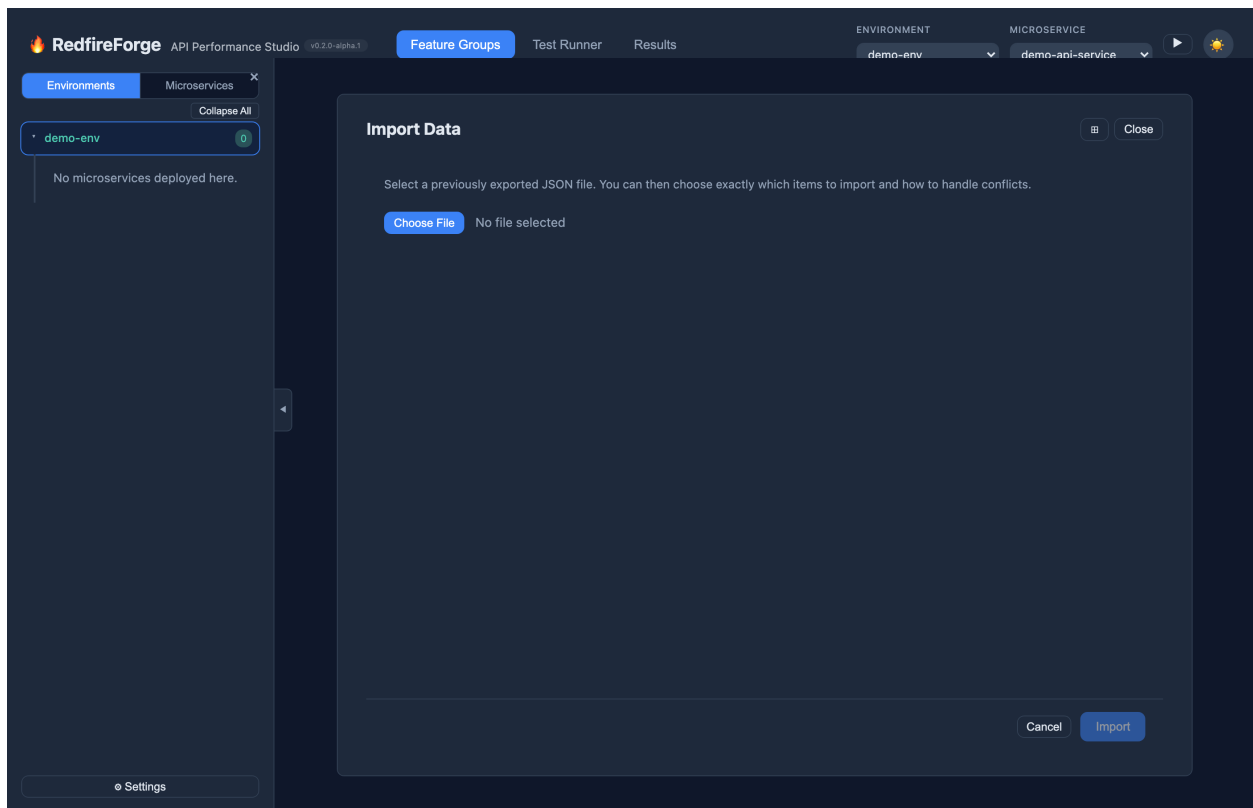


Figure 31: Import Center — choose a file and resolve conflicts

Action	Behavior
Skip	Keep existing, ignore imported item
Overwrite	Replace existing with imported item
Keep Both	Import as a new item with a new ID

14. Desktop vs Web Comparison

Feature	Desktop (Tauri)	Web (Browser)
Storage	OS AppData directory (files)	Browser localStorage
HTTP Requests	Native HTTP (no CORS issues)	Vite proxy required
File Dialogs	Native OS file dialogs	Browser file picker
Default Export Dir	Documents/RedfireForge/	Browser default
Installation	.dmg / .msi / .AppImage	npm run dev
Offline Use	Yes — fully standalone	No — needs dev server
Data Persistence	Survives browser cache clears	Lost if cache cleared

The desktop application is recommended for daily use. The web version is useful for quick UI development and testing.

15. Tips & Best Practices

Organize by Environment

Create one Feature Group per environment/microservice combination for clarity.

Use Global Auth Profiles

Define credentials once in Settings → Global Auth Profiles. Let tests inherit rather than duplicating auth config in every test.

Export Regularly

Export your configuration as a backup. Use Documents/RedfireForge/ to keep all exports organized.

Start Sequential, Then Parallel

Validate your tests in Sequential mode first. Once they pass, switch to Parallel or Ramp-up for load testing.

Monitor P95/P99

Average response time can hide outliers. Always check P95 and P99 for a realistic picture of user experience.

Use Drag-and-Drop

As your test suite grows, use drag-and-drop to restructure scenarios and tests across Feature Groups.

Prefer Desktop App

The desktop app has no CORS issues, uses native file dialogs, and stores data independently of browser cache.

16. Quick Reference

Action	Where	How
Switch views	Header bar	Click Feature Groups / Test Runner / Results
Filter by env	Sidebar	Click an Environment
Filter by service	Sidebar	Click Microservices tab
Create Feature Group	Feature Groups tab	Click '+ Add Feature Group'
Add Scenario	Inside Feature Group	Click '+ Add Scenario'
Add Test	Inside Scenario	Click '+ Add Test'
Reorder items	Feature Groups	Drag and drop
Settings	Sidebar	Click  Settings
Export config	Settings modal	Click Export Center
Import config	Settings modal	Click Import Center
Run tests	Test Runner tab	Configure → Click Run
View results	Results tab	Browse historical runs

■ RedfireForge

Fire. Measure. Validate.

Version 0.2.0